



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Disciplina: CI/CD, Pipelines e Testes Automatizados

Curso: ENGENHARIA DE INTELIGÊNCIA ARTIFICIAL E MLOPS

Instituição: PUC Minas - IEC

Professor: Wander Maia da Silva

Laboratórios Práticos - Unidade 2

Sobre estes labs

Os três labs desta unidade são **incrementais**: cada um parte do que foi construído no anterior e adiciona uma nova camada ao pipeline. Ao final do Lab 3 você terá uma pipeline completa cobrindo testes automatizados, segurança integrada, qualidade de código e observabilidade.

A aplicação usada nos três labs é a mesma: uma **calculadora de preços** em Python com funções simples de negócio, propositalmente pequena para que o foco fique inteiramente no pipeline, não na aplicação.

Lab	Âncora na apostila	O que adiciona
Lab 1	Final do Capítulo 2	Testes unitários, integração, paralelismo e cache
Lab 2	Final do Capítulo 4	Segurança: SAST, varredura de dependências e imagem Docker
Lab 3	Final do Capítulo 6	Qualidade: lint, cobertura; Observabilidade: tracing

Índice

Lab 1 — Testes Automatizados no Pipeline

- [Passo 0 — Criar o repositório e clonar localmente](#)
- [Estrutura de arquivos](#)
- [Passo 1 — Criar a aplicação](#)

- Passo 2 — Criar os testes unitários
- Passo 3 — Criar os testes de integração
- Passo 4 — Configurar o `pytest.ini`
- Passo 5 — Criar o workflow do GitHub Actions
- Como o pipeline se comporta
- Passo 6 — Fazer o push e verificar o pipeline
- Checklist de aceitação

Lab 2 — DevSecOps: Segurança no Pipeline

- Passo 1 — Adicionar dependências de segurança
- Passo 2 — Criar o Dockerfile
- Passo 3 — Criar o workflow de segurança
- Por que dois workflows separados?
- Entendendo cada camada de segurança
- O que o `fetch-depth: 0` faz no Gitleaks
- Passo 4 — Fazer o push e verificar o pipeline
- Checklist de aceitação

Lab 3 — Qualidade de Código e Observabilidade

- Passo 1 — Adicionar ferramentas de qualidade ao `requirements.txt`
- Passo 2 — Criar o arquivo de configuração do Ruff
- Passo 3 — Atualizar os testes unitários para cobertura
- Passo 4 — Criar o workflow de qualidade
- Passo 5 — Adicionar tracing ao `security.yml`
- O que o tracing captura
- Configurando os secrets necessários
- Visão consolidada do pipeline completo
- Passo 6 — Fazer o push e verificar o pipeline completo
- Checklist de aceitação

Lab 1 — Testes Automatizados no Pipeline

Contexto

Você vai criar do zero a estrutura de testes de uma aplicação Python e integrá-la a um pipeline GitHub Actions. O pipeline vai rodar testes unitários com paralelismo e cache, testes de integração com um banco de dados em container, e um smoke test condicional simulando uma verificação pós-deploy.

Ao final deste lab, o pipeline terá esta estrutura:



Passo 0 — Criar o repositório e clonar localmente

Antes de criar qualquer arquivo, o repositório precisa existir no GitHub e estar disponível na sua máquina.

1. Crie o repositório no GitHub:

Acesse github.com/new e preencha:

- **Repository name:** `calculadora-pipeline`
- **Visibility:** Public (ou Private — ambos funcionam com GitHub Actions)
- Marque **"Add a README file"** para que o repositório já inicie com um commit

Clique em **"Create repository"**.

2. Clone o repositório localmente:

```
git clone https://github.com/<seu-usuario>/calculadora-pipeline.git
cd calculadora-pipeline
```

3. Crie a estrutura de diretórios:

```
mkdir -p src tests/unit tests/integration .github/workflows
touch src/__init__.py tests/__init__.py tests/unit/__init__.py
tests/integration/__init__.py
```

Estrutura de arquivos

Ao final do lab, seu repositório terá esta organização:

```
calculadora-pipeline/
├── src/
│   ├── __init__.py
│   ├── calculadora.py
│   └── repositorio.py
├── tests/
│   ├── __init__.py
│   ├── unit/
│   │   ├── __init__.py
│   │   └── test_calculadora.py
│   └── integration/
│       ├── __init__.py
│       └── test_repositorio.py
├── requirements.txt
├── .github/
│   └── workflows/
│       └── tests.yml
```

Passo 1 — Criar a aplicação

Crie o diretório do projeto e os arquivos da aplicação.

src/calculadora.py - funções de negócio puras:

```
def calcular_desconto(preco: float, percentual: float) -> float:
    if percentual < 0 or percentual > 100:
        raise ValueError("Percentual deve estar entre 0 e 100")
    return round(preco * (1 - percentual / 100), 2)

def calcular_imposto(preco: float, aliquota: float) -> float:
    if aliquota < 0:
        raise ValueError("Alíquota não pode ser negativa")
    return round(preco * (aliquota / 100), 2)

def calcular_preco_final(
    preco_base: float, desconto: float, aliquota_imposto: float
) -> float:
    preco_com_desconto = calcular_desconto(preco_base, desconto)
    imposto = calcular_imposto(preco_com_desconto, aliquota_imposto)
    return round(preco_com_desconto + imposto, 2)
```

src/repositorio.py - acesso ao banco de dados:

```
def criar_tabela(conn):
    with conn.cursor() as cur:
        cur.execute("""
            CREATE TABLE IF NOT EXISTS precos (
                id SERIAL PRIMARY KEY,
                produto VARCHAR(100) NOT NULL,
                preco NUMERIC(10, 2) NOT NULL
            )
        """)
    conn.commit()

def salvar_preco(conn, produto: str, preco: float):
    with conn.cursor() as cur:
        cur.execute(
            "INSERT INTO precos (produto, preco) VALUES (%s, %s)",
            (produto, preco)
        )
    conn.commit()

def buscar_preco(conn, produto: str) -> float | None:
    with conn.cursor() as cur:
        cur.execute("SELECT preco FROM precos WHERE produto = %s",
            (produto,))
        resultado = cur.fetchone()
        return float(resultado[0]) if resultado else None
```

`src/__init__.py` — vazio, apenas para tornar o diretório um pacote Python.

`requirements.txt`:

```
pytest==8.2.2
pytest-xdist==3.5.0
pytest-cov==5.0.0
psycpg2-binary==2.9.9
```

Passo 2 — Criar os testes unitários

Os testes unitários validam as funções de `calculadora.py` em isolamento total, sem banco de dados, sem rede.

`tests/unit/test_calculadora.py`:

```

import pytest
from src.calculadora import calcular_desconto, calcular_imposto,
calcular_preco_final

@pytest.mark.unit
class TestCalcularDesconto:
    def test_desconto_normal(self):
        assert calcular_desconto(100.0, 10) == 90.0

    def test_desconto_zero(self):
        assert calcular_desconto(100.0, 0) == 100.0

    def test_desconto_total(self):
        assert calcular_desconto(100.0, 100) == 0.0

    def test_desconto_percentual_negativo(self):
        with pytest.raises(ValueError, match="entre 0 e 100"):
            calcular_desconto(100.0, -5)

    def test_desconto_percentual_acima_de_100(self):
        with pytest.raises(ValueError):
            calcular_desconto(100.0, 110)

@pytest.mark.unit
class TestCalcularImposto:
    def test_imposto_normal(self):
        assert calcular_imposto(100.0, 15) == 15.0

    def test_imposto_zero(self):
        assert calcular_imposto(100.0, 0) == 0.0

    def test_aliquota_negativa(self):
        with pytest.raises(ValueError, match="negativa"):
            calcular_imposto(100.0, -1)

@pytest.mark.unit
class TestCalcularPrecoFinal:
    def test_preco_final_completo(self):
        # 100 - 10% desconto = 90; 90 + 10% imposto = 99
        assert calcular_preco_final(100.0, 10, 10) == 99.0

    def test_sem_desconto_sem_imposto(self):
        assert calcular_preco_final(200.0, 0, 0) == 200.0

```

Passo 3 — Criar os testes de integração

Os testes de integração verificam que as funções de `repositorio.py` realmente salvam e leem dados do banco Postgres. Eles precisam de uma conexão real com o banco que será provida pelo

CI como um container de serviço.

tests/integration/test_repositorio.py:

```
import os
import pytest
import psycopg2
from src.repositorio import criar_tabela, salvar_preco, buscar_preco

@pytest.fixture(scope="module")
def conn():
    database_url = os.environ["DATABASE_URL"]
    connection = psycopg2.connect(database_url)
    criar_tabela(connection)
    yield connection
    connection.close()

@pytest.mark.integration
def test_salvar_e_buscar_preco(conn):
    salvar_preco(conn, "notebook", 3500.00)
    resultado = buscar_preco(conn, "notebook")
    assert resultado == 3500.00

@pytest.mark.integration
def test_produto_inexistente_retorna_none(conn):
    resultado = buscar_preco(conn, "produto_que_nao_existe")
    assert resultado is None

@pytest.mark.integration
def test_buscar_preco_apos_multiplos_saves(conn):
    salvar_preco(conn, "mouse", 150.00)
    salvar_preco(conn, "teclado", 250.00)
    assert buscar_preco(conn, "mouse") == 150.00
    assert buscar_preco(conn, "teclado") == 250.00
```

Sobre a fixture `conn`: o `scope="module"` significa que a conexão com o banco é aberta uma única vez para todos os testes do arquivo, não uma por teste. Isso economiza tempo e evita erros de conexões simultâneas.

Passo 4 — Configurar o `pytest.ini`

Crie um arquivo `pytest.ini` na raiz do projeto para registrar os marcadores customizados e evitar avisos do pytest:

```
[pytest]
markers =
    unit: testes unitários sem dependências externas
    integration: testes que precisam de infraestrutura (banco, fila,
etc.)
```

Passo 5 — Criar o workflow do GitHub Actions

Crie o arquivo `.github/workflows/tests.yml`:

```
name: Testes

on:
  push:
    branches: [main]
  pull_request:

jobs:
  unit-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Cache de dependências
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${ runner.os }-pip-${ hashFiles('requirements.txt') }
        }}

      - name: Restaurar dependências
        run: pip restore-keys: ${ runner.os }-pip-

      - name: Instalar dependências
        run: pip install -r requirements.txt

      - name: Executar testes unitários
        run: pytest tests/unit/ -n auto -m unit --tb=short --
junitxml=report-unit.xml

      - name: Publicar relatório
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: report-unit
          path: report-unit.xml

  integration-tests:
    runs-on: ubuntu-latest
    needs: unit-tests
```



```

needs: integration-tests

services:
  postgres:
    image: postgres:15
    env:
      POSTGRES_DB: testdb
      POSTGRES_USER: test
      POSTGRES_PASSWORD: test
    ports:
      - 5432:5432
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5

steps:
  - uses: actions/checkout@v4

  - uses: actions/setup-python@v5
    with:
      python-version: "3.12"

  - name: Cache de dependências
    uses: actions/cache@v4
    with:
      path: ~/.cache/pip
      key: ${ runner.os }-pip-${ hashFiles('requirements.txt') }
      restore-keys: ${ runner.os }-pip-

  - name: Instalar dependências
    run: pip install -r requirements.txt

  - name: Executar testes de integração
    run: pytest tests/integration/ -m integration --tb=short --
junitxml=report-integration.xml
    env:
      DATABASE_URL: postgresql://test:test@localhost/testdb

  - name: Publicar relatório
    uses: actions/upload-artifact@v4
    if: always()
    with:
      name: report-integration
      path: report-integration.xml

smoke-test:
  runs-on: ubuntu-latest
  needs: integration-tests
  if: github.ref == 'refs/heads/main'

  steps:
    - uses: actions/checkout@v4

```

```

- uses: actions/setup-python@v5
  with:
    python-version: "3.12"

- name: Instalar dependências
  run: pip install -r requirements.txt

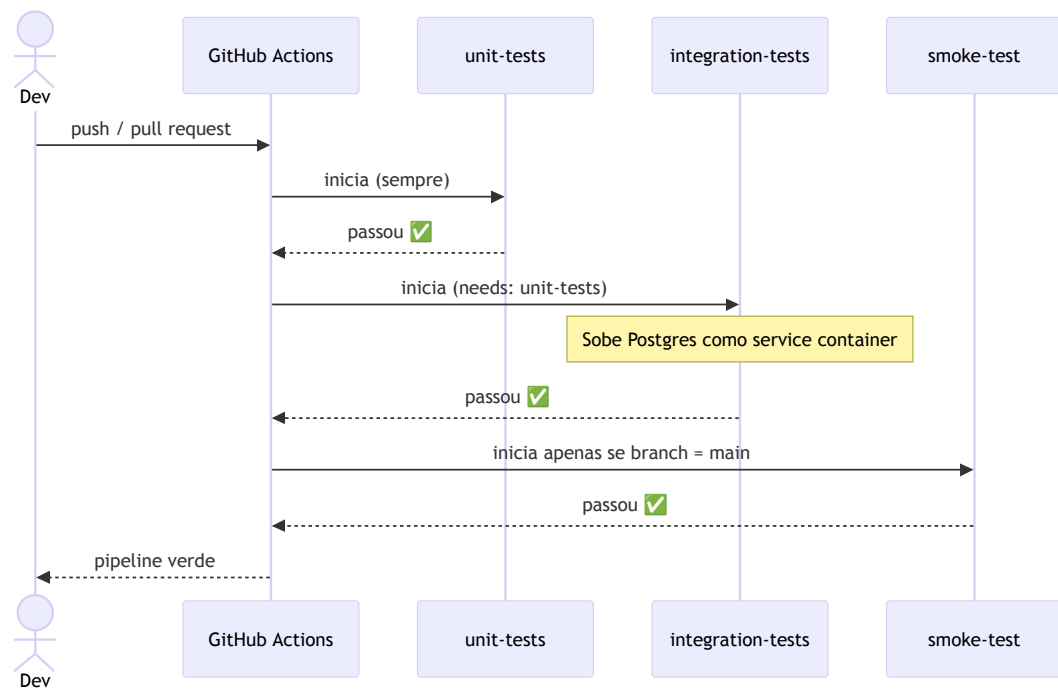
- name: Smoke test — verificar integridade do módulo
  run: |
    python -c "
      from src.calculadora import calcular_desconto,
calcular_imposto, calcular_preco_final
      assert calcular_desconto(100, 10) == 90.0
      assert calcular_imposto(100, 15) == 15.0
      assert calcular_preco_final(100, 10, 10) == 99.0
      print('Smoke test passou — módulo operacional')
    "

```

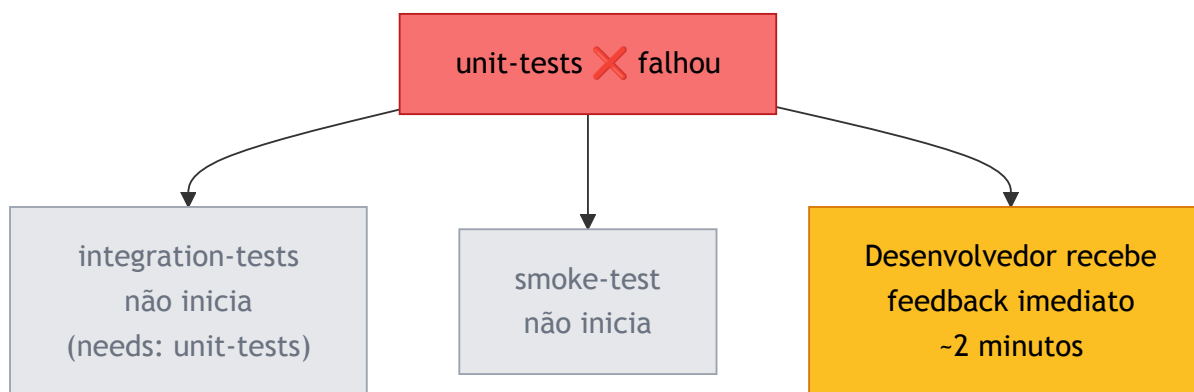
Nota — smoke test simplificado: a apostila (Cap 1.5) define smoke test como uma verificação rápida de que o sistema está operacional **após um deploy**, com exemplo usando `curl --fail http://localhost:8080/health`. Neste lab não há serviço HTTP rodando, a aplicação é um conjunto de funções puras. Por isso o smoke test valida a importação e execução das funções diretamente, simulando a verificação pós-deploy. Em projetos reais com uma API ou serviço web, o padrão da apostila se aplica diretamente: o smoke test faria `curl` contra o endpoint `/health` da aplicação deployada em staging.

Equivalente em Azure Pipelines: o mesmo fluxo usa `stages` com `dependsOn` para o sequenciamento. O job de integração usa `services` dentro do `container`. O smoke test usa `condition: and(succeeded(), eq(variables['Build.SourceBranch'], 'refs/heads/main'))`. O cache é configurado com `task: Cache@2` e chave `'pip | "$(Agent.OS)" | requirements.txt'`.

Como o pipeline se comporta



O que acontece se os unitários falharem?



O **needs** garante que testes mais lentos e caros nunca rodam quando os mais rápidos já falharam, o feedback chega em minutos, não em horas.

Passo 6 — Fazer o push e verificar o pipeline

Com todos os arquivos criados, faça o commit e envie para o GitHub:

```
# Adicionar todos os arquivos novos ao staging
git add .

# Verificar o que será commitado
git status

# Criar o commit
git commit -m "lab1: estrutura de testes e pipeline GitHub Actions"

# Enviar para o GitHub
git push origin main
```

Após o push, acesse **github.com/<seu-usuario>/calculadora-pipeline/actions**, o workflow **Testes** deve aparecer em execução automaticamente.

Para simular um pull request e ver o **smoke-test** sendo pulado:

```
# Criar uma branch de feature
git checkout -b feature/meu-teste

# Fazer qualquer alteração pequena (ex: adicionar um comentário)
git add .
git commit -m "test: verificar comportamento do pipeline em PR"
git push origin feature/meu-teste
```

Abra o PR no GitHub seguindo estes passos:

1. Acesse o repositório no GitHub
2. Clique em **Compare & pull request** (o banner amarelo que aparece após o push) — ou acesse a aba **Pull requests** → **New pull request**
3. Selecione **feature/meu-teste** como branch de origem e **main** como destino
4. Clique em **Create pull request**

Na aba **Actions**, abra a execução disparada pelo PR e observe que os jobs **unit-tests** e **integration-tests** aparecem e executam, mas o **smoke-test** fica com status **"skipped"** » — ele só roda quando o código chega na **main**.

Checklist de aceitação

Antes de executar os itens abaixo, confirme que as dependências estão instaladas num ambiente virtual ativo. Se ainda não criou o venv, faça agora:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Por que o venv é necessário? Ubuntu e Debian a partir do Python 3.12 bloqueiam instalações diretas no Python do sistema (`externally-managed-environment`). O venv cria um ambiente isolado onde o `pip install` funciona normalmente sem risco de quebrar pacotes do OS.

Para ativar o venv em sessões futuras, basta rodar `source .venv/bin/activate` na raiz do projeto. Confirme que está ativo com `which pytest`, o caminho deve conter `.venv`.

Se o comando `pytest -n auto` retornar `unrecognized arguments: -n auto` mesmo após o `pip install`, o `pytest-xdist` não está instalado no ambiente ativo. Verifique se o venv está ativado antes de rodar o pytest.

Antes de avançar para o Lab 2, verifique cada item:

Código e testes

- ☐ `src/calculadora.py` contém as três funções com validação de entrada
- ☐ `src/repositorio.py` contém `criar_tabela`, `salvar_preco` e `buscar_preco`
- ☐ `tests/unit/test_calculadora.py` tem pelo menos 8 casos de teste marcados com `@pytest.mark.unit`
- ☐ `tests/integration/test_repositorio.py` tem pelo menos 3 casos marcados com `@pytest.mark.integration`
- ☐ `pytest.ini` registra os marcadores `unit` e `integration`
- ☐ Rodando `pytest tests/unit/ -m unit` localmente: todos os testes passam
- ☐ Rodando `pytest tests/unit/ -n auto` localmente: execução paralela ativa (saída mostra workers)

Pipeline

- ☐ Job `unit-tests` usa `actions/cache@v4` — na segunda execução, o passo de cache mostra "Cache restored"

Para validar, faça dois pushes consecutivos sem alterar o `requirements.txt`. Na aba **Actions**, abra a segunda execução, clique no job `unit-tests` e expanda o step **Cache de dependências**. Na primeira execução ele exibe `Cache not found`; na segunda exibe `Cache restored from key: ...`, e o step de instalação de dependências completa em segundos em vez de dezenas de segundos.

- ☐ Job `integration-tests` declara `needs: unit-tests` e o bloco `services.postgres`
- ☐ Job `integration-tests` recebe `DATABASE_URL` via `env`
- ☐ Job `smoke-test` tem `if: github.ref == 'refs/heads/main'` — não aparece em PRs de outras branches

Para validar, abra um PR de uma branch diferente da `main` (ex: a `feature/meu-teste` criada no Passo 6). Na aba **Actions**, abra a execução disparada pelo PR e observe que os jobs `unit-tests` e `integration-tests` aparecem, mas o `smoke-test` não é listado. Em seguida, faça merge do PR na `main` — na execução disparada pelo push na `main`, o `smoke-test` aparecerá e será executado.

- ❑ Relatórios JUnit (`report-unit.xml`, `report-integration.xml`) aparecem como artefatos na execução
- ❑ Ao forçar falha em um teste unitário, os jobs `integration-tests` e `smoke-test` ficam com status "skipped"

Para validar este item, edite `tests/unit/test_calculadora.py` e troque um valor esperado por um errado:

```
def test_desconto_normal():
    assert calcular_desconto(100.0, 10) == 999.0 # valor errado propositalmente
```

Faça commit e push. Na aba **Actions** do GitHub, observe que `unit-tests` fica **failed** ✖ e os jobs `integration-tests` e `smoke-test` ficam **skipped** ⏸. Após validar, desfaça a alteração e faça um novo commit para restaurar o pipeline verde.

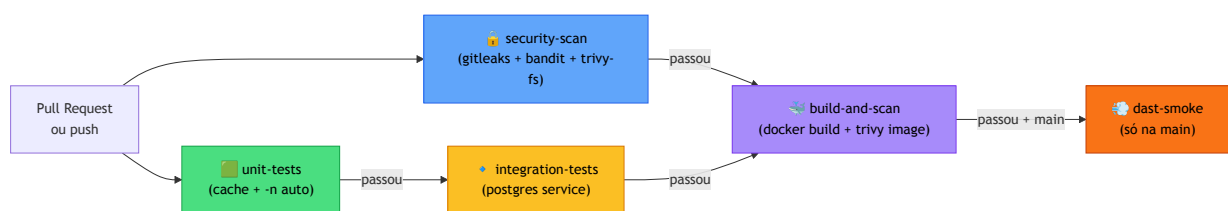
Lab 2 — DevSecOps: Segurança no Pipeline

Contexto

Neste lab você adiciona três camadas de segurança ao pipeline construído no Lab 1. O ponto de partida é o `tests.yml`, que já existe, você vai criar um segundo workflow dedicado a segurança que roda em paralelo com os testes.

A lógica é simples: segurança e testes são verificações independentes. Não faz sentido esperar uma terminar para a outra começar. O build da imagem Docker só acontece depois que **ambas** passaram.

Ao final deste lab, o pipeline terá esta estrutura:



Passo 1 — Adicionar dependências de segurança

Adicione as ferramentas de SAST e varredura ao `requirements.txt`:

```
pytest==8.2.2
pytest-xdist==3.5.0
pytest-cov==5.0.0
psycpg2-binary==2.9.9
bandit==1.7.9
```

Passo 2 — Criar o Dockerfile

O job `build-and-scan` precisa de uma imagem para construir e varrer. Crie o `Dockerfile` na raiz do projeto:

```
FROM python:3.12-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src/

CMD ["python", "-c", "from src.calculadora import calcular_preco_final; print('app ok')"]
```

Por que `python:3.12-slim`? A imagem `slim` remove ferramentas de desenvolvimento e documentação que não são necessárias em produção, reduzindo a superfície de ataque em relação à imagem completa `python:3.12`. Para menor superfície ainda, `python:3.12-alpine` é uma opção, mas pode gerar incompatibilidades com pacotes binários como `psycopg2`.

Passo 3 — Criar o workflow de segurança

Crie o arquivo `.github/workflows/security.yml`. Este workflow é independente do `tests.yml` e roda em paralelo com ele.

```
name: Segurança

on:
  push:
    branches: [main]
  pull_request:

jobs:
  security-scan:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Varredura de secrets com Gitleaks
        uses: gitleaks/gitleaks-action@v2
        env:
          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }

      - uses: actions/setup-python@v5
```

```

    with:
      python-version: "3.12"

- name: Cache de dependências
  uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${ runner.os }}-pip-${ hashFiles('requirements.txt')
  }}

    restore-keys: ${ runner.os }}-pip-

- name: Instalar dependências
  run: pip install -r requirements.txt

- name: SAST com Bandit
  run: bandit -r src/ -ll -f json -o bandit-report.json

- name: Publicar relatório Bandit
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: bandit-report
    path: bandit-report.json

- name: Varredura de dependências com Trivy
  uses: aquasecurity/trivy-action@master
  with:
    scan-type: fs
    scan-ref: .
    scanners: vuln
    severity: CRITICAL,HIGH
    exit-code: "1"
    ignore-unfixed: true
    format: table

build-and-scan:
  runs-on: ubuntu-latest
  needs: security-scan

  steps:
    - uses: actions/checkout@v4

    - name: Build da imagem Docker
      run: docker build -t calculadora-app:${ github.sha }} .

    - name: Varredura da imagem com Trivy
      uses: aquasecurity/trivy-action@master
      with:
        image-ref: calculadora-app:${ github.sha }}
        severity: CRITICAL,HIGH
        exit-code: "1"
        ignore-unfixed: true
        format: table

dast-smoke:

```



```

runs-on: ubuntu-latest
needs: build-and-scan
if: github.ref == 'refs/heads/main'

steps:
  - uses: actions/checkout@v4

  - name: Reconstruir imagem para o smoke test
    run: docker build -t calculadora-app:${{ github.sha }} .
    # Cada job roda num runner efêmero separado – a imagem
    # construída em
    # build-and-scan não persiste entre jobs, por isso
    # reconstruímos aqui.

  - name: Subir aplicação em contêiner
    run: docker run -d --name app-teste calculadora-app:${{
github.sha }} sleep 30

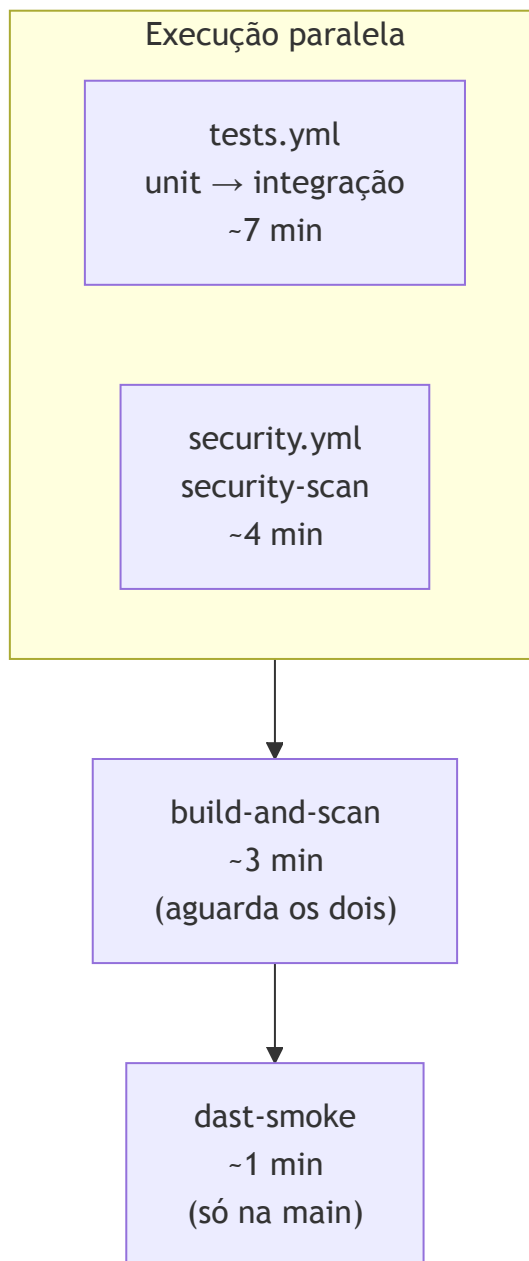
  - name: DAST smoke – verificar que o contêiner está operacional
    run: |
      docker exec app-teste python -c "
        from src.calculadora import calcular_desconto,
calcular_imposto
        assert calcular_desconto(200, 25) == 150.0
        assert calcular_imposto(200, 10) == 20.0
        print('DAST smoke passou – aplicação operacional no
contêiner')
      "

  - name: Parar contêiner de teste
    if: always()
    run: docker stop app-teste && docker rm app-teste

```

Equivalente em Azure Pipelines: o workflow de segurança vira um stage separado com `dependsOn: []` (roda em paralelo com o stage de testes). O Gitleaks é executado via `script: ./gitleaks detect --source . --exit-code 1` após download do binário. O Bandit usa `script: bandit -r src/ -ll`. O Trivy usa a task da extensão do Marketplace ou instalação via script. O `build-and-scan` vira outro stage com `dependsOn: [Seguranca, Testes]` para esperar ambos.

Por que dois workflows separados?



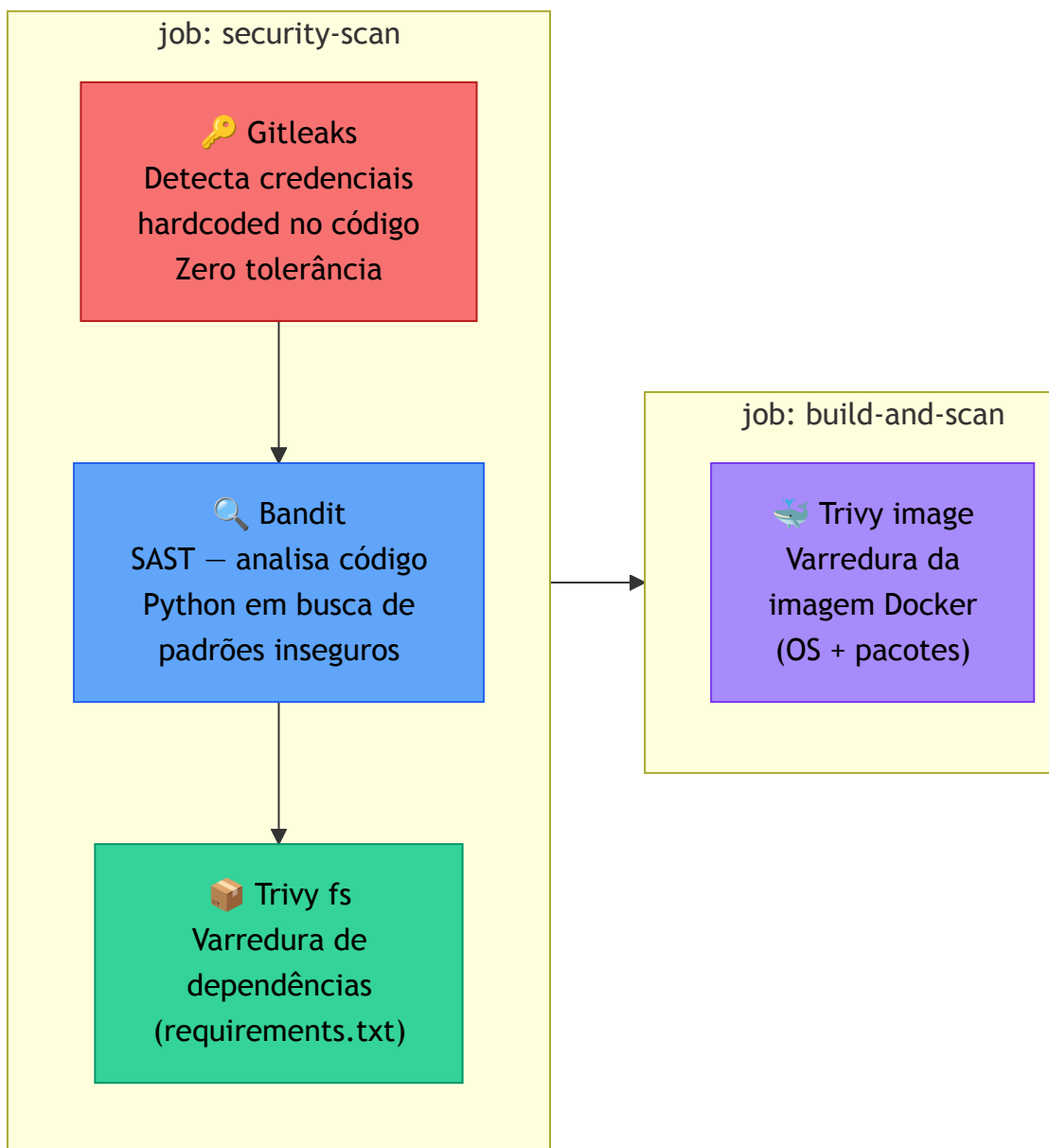
Com os dois workflows em paralelo, o tempo total até o `build-and-scan` é determinado pelo mais lento, geralmente os testes de integração. O scan de segurança roda ao mesmo tempo, sem custo adicional de tempo.

Se você colocasse segurança em sequência depois dos testes, somaria os tempos: $7 + 4 = 11$ minutos. Em paralelo: $\max(7, 4) = 7$ minutos.

Limitação importante — `needs` não atravessa workflows: o `needs` do GitHub Actions só funciona entre jobs do **mesmo arquivo de workflow**. O `build-and-scan` dentro de `security.yml` consegue aguardar o `security-scan` (mesmo arquivo), mas **não consegue aguardar** o `integration-tests` de `tests.yml` (arquivo diferente). Na prática, assim que `security-scan` passa, o `build-and-scan` inicia, independente do estado dos testes. Isso é aceitável porque os dois workflows são verificações paralelas independentes: se os testes falharem, o commit será bloqueado pelo `tests.yml`; se a segurança falhar, será bloqueado pelo `security.yml`. O diagrama no início deste lab representa a **intenção lógica** do pipeline, não uma dependência técnica via `needs`. Quando essa coordenação

rígida entre workflows é necessária, a alternativa é consolidar tudo em um único arquivo `.yaml`, usando `needs` entre jobs.

Entendendo cada camada de segurança



Ferramenta	O que detecta	Bloqueia em
Gitleaks	Tokens, senhas e chaves no código-fonte	Qualquer ocorrência
Bandit	Vulnerabilidades no código Python (SQLi, uso de funções inseguras, etc.)	Severidade MEDIUM ou HIGH (-11)
Trivy fs	CVEs nas dependências do <code>requirements.txt</code>	CRITICAL e HIGH com correção disponível
Trivy image	CVEs no SO base e pacotes da imagem Docker	CRITICAL e HIGH com correção disponível

O que o `fetch-depth: 0` faz no Gitleaks

Por padrão, o `actions/checkout` clona apenas o último commit (`fetch-depth: 1`). O Gitleaks precisa do histórico completo para detectar credenciais que foram adicionadas e depois removidas, pois um segredo apagado num commit ainda existe no histórico e pode ser extraído.

```
- uses: actions/checkout@v4
  with:
    fetch-depth: 0    # histórico completo — necessário para o
    Gitleaks
```

Passo 4 — Fazer o push e verificar o pipeline

Com os novos arquivos criados, faça o commit e envie para o GitHub:

```
git add Dockerfile requirements.txt .github/workflows/security.yml
git status
git commit -m "lab2: segurança — gitleaks, bandit, trivy e build
docker"
git push origin main
```

Após o push, acesse a aba **Actions** do repositório. Você verá dois workflows iniciando simultaneamente para o mesmo commit: `Testes` e `Segurança`. Observe que `build-and-scan` só aparece após `security-scan` concluir com sucesso.

Checklist de aceitação

Antes de avançar para o Lab 3, verifique cada item:

Arquivos

- ☐ `Dockerfile` existe na raiz com imagem base `python:3.12-slim`
- ☐ `bandit` está listado no `requirements.txt`
- ☐ `.github/workflows/security.yml` existe com os jobs `security-scan`, `build-and-scan` e `dast-smoke`

Job `security-scan`

- ☐ `actions/checkout` usa `fetch-depth: 0`
- ☐ Step do Gitleaks usa `gitleaks/gitleaks-action@v2` com `GITHUB_TOKEN`
- ☐ Step do Bandit usa `-ll` (bloqueia apenas MEDIUM e HIGH) e exporta `bandit-report.json`
- ☐ Step do Bandit usa `if: always()` no upload do artefato — relatório é publicado mesmo quando o Bandit encontra problemas
- ☐ Step do Trivy fs usa `ignore-unfixed: true` e `severity: CRITICAL,HIGH`

Job `build-and-scan`

- ☐ Declara `needs: security-scan`
- ☐ Faz `docker build` referenciando `${{ github.sha }}` como tag
- ☐ Trivy image usa `exit-code: "1"` — falha o job se encontrar vulnerabilidades

Job `dast-smoke`

- ☐ Declara `needs: build-and-scan` e `if: github.ref == 'refs/heads/main'`
- ☐ O step de parada do contêiner usa `if: always()` — limpa o ambiente mesmo em caso de falha
- ☐ Ao rodar em um PR de feature branch, o job `dast-smoke` fica com status "skipped"

Comportamento do pipeline

- ☐ Os workflows `tests.yml` e `security.yml` aparecem como execuções paralelas na aba Actions do repositório
- ☐ Relatório `bandit-report.json` aparece como artefato na execução do `security-scan`
- ☐ Ao adicionar um token falso hardcoded no código (ex: `api_key = "AKIAIOSFODNN7EXAMPLE3"`), o Gitleaks bloqueia o pipeline

Para validar, adicione uma linha com credencial falsa em qualquer arquivo Python do projeto, por exemplo no final de `src/calculadora.py`:

```
api_key = "AKIAIOSFODNN7EXAMPLE3" # simulando credencial
hardcoded
```

Por que não usar `AKIAIOSFODNN7EXAMPLE`? Esse valor é o exemplo canônico da documentação oficial da AWS e está na allowlist do Gitleaks. Ele é ignorado propositalmente para evitar falso positivo em tutoriais. Use uma variação como a acima, que segue o padrão real de chave AWS (`AKIA` + 16 caracteres alfanuméricos maiúsculos) mas não está na allowlist.

Faça commit e push. Na aba **Actions**, observe que o job `security-scan` do workflow `security.yml` falha x com mensagem indicando que um secret foi detectado. Após validar, **remova a linha imediatamente** e faça um novo commit, nunca deixe credenciais no histórico do repositório, mesmo que falsas.

Lab 3 — Qualidade de Código e Observabilidade

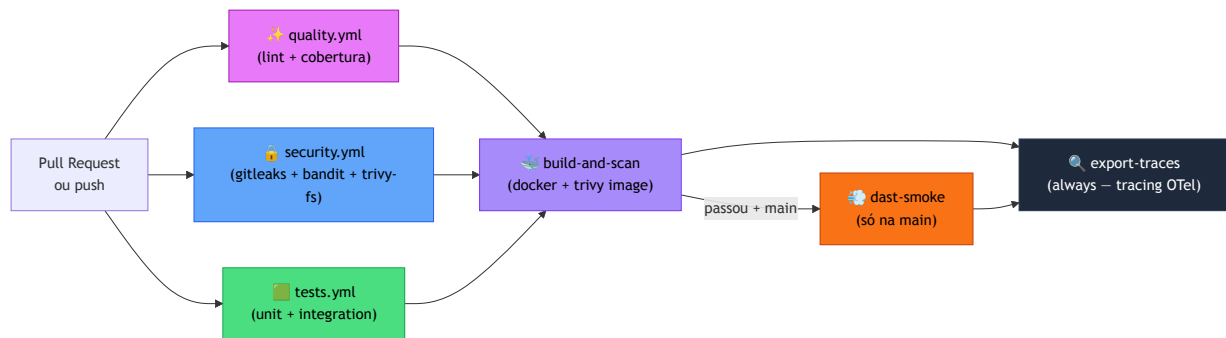
Contexto

Este lab fecha o ciclo da unidade. Você vai adicionar duas camadas finais ao pipeline: **qualidade de código** (lint, formatação e cobertura mínima de testes) e **observabilidade** (tracing do próprio pipeline com OpenTelemetry).

A qualidade entra como um terceiro workflow paralelo junto com testes e segurança. O tracing é adicionado como um job final que sempre roda, independente do resultado dos outros, e exporta a

execução como um trace navegável.

Ao final deste lab, o pipeline completo da unidade terá esta estrutura:



Escopo deste lab — o que foi omitido intencionalmente:

SonarQube (Cap 5.1): o SonarQube é a ferramenta de referência para quality gates completos, bugs, code smells, vulnerabilidades e cobertura num único painel. Ele não está neste lab porque requer um servidor externo (SonarQube Cloud ou instância própria) e um `SONAR_TOKEN` configurado, o que tornaria o setup muito dependente de infraestrutura externa. O lab cobre os mesmos objetivos de qualidade (lint, formatação e cobertura mínima) com ferramentas locais. Para integrar o SonarQube, consulte o Cap 5.1 da apostila, o workflow de exemplo é direto e se encaixaria como um job adicional no `quality.yml`.

Prometheus, Grafana e Loki (Cap 6.2 e 6.3): métricas e logs centralizados do pipeline exigem infraestrutura de observabilidade rodando externamente (um OTEL Collector, instâncias de Prometheus e Loki acessíveis pelo runner do GitHub). O lab cobre o pilar de **traces** (Cap 6.4), que tem a melhor relação entre conceito aprendido e facilidade de setup. A stack completa de métricas e logs é o exercício natural de aprofundamento após este lab — o `docker-compose.observability.yml` descrito no Cap 6.2 pode ser executado localmente como complemento.

Passo 1 — Adicionar ferramentas de qualidade ao `requirements.txt`

```
pytest==8.2.2
pytest-xdist==3.5.0
pytest-cov==5.0.0
psycpg2-binary==2.9.9
bandit==1.7.9
ruff==0.4.10
```

O `black` não é adicionado ao `requirements.txt` porque o Ruff, a partir da versão 0.1, incorpora formatação compatível com Black — basta usar `ruff format`. Uma dependência a menos, o mesmo resultado.

Passo 2 — Criar o arquivo de configuração do Ruff

Crie o arquivo `ruff.toml` na raiz do projeto para definir as regras de lint e formatação:

```
[lint]
select = ["E", "F", "W"]
ignore = []

[format]
quote-style = "double"
indent-style = "space"
```

As regras **E** (erros de estilo PEP 8), **F** (erros Pyflakes: imports não usados, variáveis indefinidas) e **W** (warnings) cobrem os problemas mais comuns sem gerar ruído excessivo de alertas.

Passo 3 — Atualizar os testes unitários para cobertura

O job de qualidade vai exigir cobertura mínima de 80%. Para que o relatório seja preciso, o `pytest-cov` precisa saber quais arquivos medir. Confirme que o `pytest.ini` existente está correto e adicione a configuração de cobertura:

pytest.ini (atualizado):

```
[pytest]
markers =
    unit: testes unitários sem dependências externas
    integration: testes que precisam de infraestrutura (banco, fila,
etc.)

[coverage:run]
source = src.calculadora
omit = tests/*
```

Por que `src.calculadora` e não `src`? O diretório `src/` contém dois módulos: `calculadora.py` e `repositorio.py`. Os testes unitários cobrem apenas `calculadora.py`, `repositorio.py` exige banco de dados e é coberto pelos testes de integração em `tests.yml`. Usar `source = src` mediria os dois módulos, fazendo `repositorio.py` contribuir com 0% e derrubando a média abaixo de 80% mesmo com todos os testes unitários passando. `src.calculadora` usa a **notação de módulo Python** (com ponto, não barra), que é exigida pelo coverage para identificar um módulo específico dentro de um pacote. A notação de caminho `src/calculadora` não funciona aqui.

Passo 4 — Criar o workflow de qualidade

Crie o arquivo `.github/workflows/quality.yml`:

```
name: Qualidade

on:
```

```

push:
  branches: [main]
pull_request:

jobs:
  lint-and-format:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Cache de dependências
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${ runner.os }-pip-${ hashFiles('requirements.txt') }

      - name: Instalar dependências
        run: pip install -r requirements.txt

      - name: Lint com Ruff
        run: ruff check src/ tests/

      - name: Verificar formatação com Ruff
        run: ruff format --check src/ tests/

  test-coverage:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Cache de dependências
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${ runner.os }-pip-${ hashFiles('requirements.txt') }

      - name: Instalar dependências
        run: pip install -r requirements.txt

      - name: Executar testes com cobertura
        run: >
          pytest tests/unit/
          -m unit

```



```

--cov=src.calculadora
--cov-report=xml
--cov-report=html
--cov-fail-under=80
--tb=short

- name: Publicar relatório HTML de cobertura
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: coverage-report
    path: htmlcov/

- name: Publicar relatório XML de cobertura
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: coverage-xml
    path: coverage.xml

```

Equivalente em Azure Pipelines: o workflow de qualidade vira um stage com dois jobs paralelos: `LintFormat` e `TestCoverage`. O `ruff check` e `ruff format --check` rodam via `script`. O relatório XML de cobertura é publicado com `task: PublishCodeCoverageResults@2`, que exibe a cobertura nativamente na aba do pipeline no Azure DevOps sem precisar de action externa.

Passo 5 — Adicionar tracing ao `security.yml`

O job de tracing precisa ser o último a rodar e deve sempre executar, mesmo quando outros jobs falharam. Adicione o job `export-traces` ao final do `security.yml` existente:

```

export-traces:
  runs-on: ubuntu-latest
  needs: [build-and-scan, dast-smoke]
  if: always()
  steps:
    - uses: inception-health/otel-export-trace-action@v1
      with:
        otelEndpoint: grpc://${{ secrets.OTEL_ENDPOINT }}:4317
        otelHeaders: "Authorization=Bearer ${{ secrets.OTEL_TOKEN }}"

        otelServiceName: calculadora-pipeline
        githubToken: ${{ secrets.GITHUB_TOKEN }}
        runId: ${{ github.run_id }}

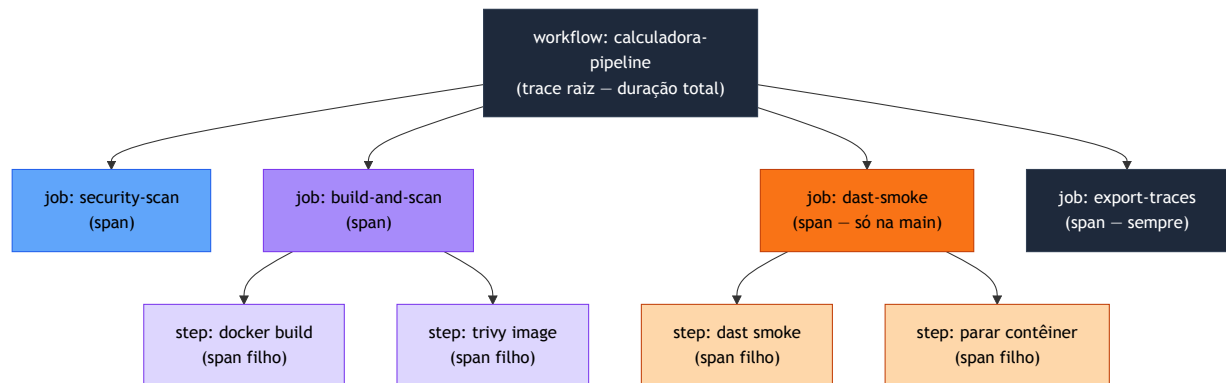
```

Por que o tracing fica no `security.yml` e não num workflow separado? O job `export-traces` precisa ser o último a rodar no ciclo. Como o `security.yml` já contém o `build-and-scan` e o `dast-smoke`, os jobs mais tardios do pipeline, é o lugar natural para o job de

encerramento. Um workflow separado não conseguiria aguardar jobs de outros workflows com `needs`.

O que o tracing captura

Cada execução do `security.yml` gera um trace com esta estrutura no backend OTel (Jaeger ou Grafana Tempo):



Com esse trace você consegue responder perguntas que métricas agregadas não respondem:

- "Por que a execução de hoje levou 3 minutos a mais?" → o span `trivy image` mostra que o pull da imagem base demorou, provavelmente cold cache no runner.
- "O dast-smoke está consumindo tempo proporcional?" → compare os spans lado a lado em execuções consecutivas.
- "Qual job foi o gargalo desta semana?" → sem precisar abrir cada log individualmente.

Configurando os secrets necessários

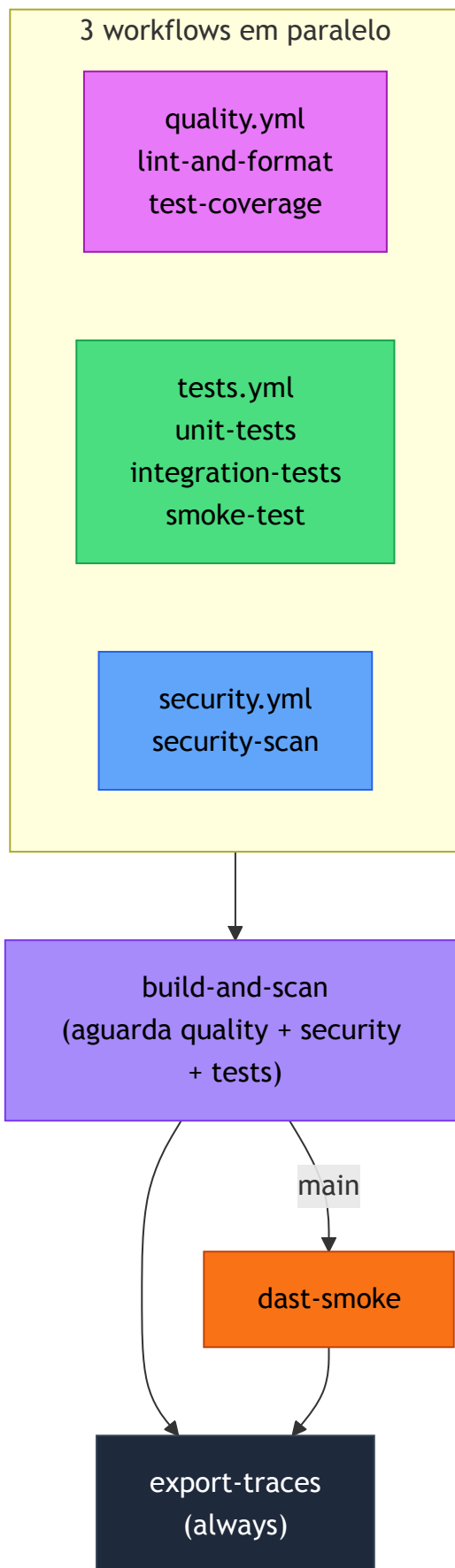
Para o tracing funcionar, o repositório precisa de dois secrets configurados em **Settings** → **Secrets and variables** → **Actions**:

Secret	Valor	Necessário para
<code>OTEL_ENDPOINT</code>	hostname ou IP do OTel Collector	Exportar traces
<code>OTEL_TOKEN</code>	token de autenticação do collector	Autenticação

Para fins de laboratório: se não houver um OTel Collector disponível, substitua o step de `export-traces` por um step que apenas imprime o `run_id` e a duração simulada. O objetivo do lab é entender a estrutura e o propósito do tracing, a integração real com o backend é um exercício de infraestrutura complementar.

```
export-traces:
  runs-on: ubuntu-latest
  needs: [build-and-scan, dast-smoke]
  if: always()
  steps:
    - name: Simular exportação de trace
      run: |
        echo "Run ID: ${github.run_id}"
        echo "Workflow: ${github.workflow}"
        echo "Ref: ${github.ref}"
        echo "SHA: ${github.sha}"
        echo "Trace exportado (modo simulado – configure
OTEL_ENDPOINT para backend real)"
```

Visão consolidada do pipeline completo



Os três workflows disparam ao mesmo tempo no mesmo evento (push ou pull_request). Como rodam em paralelo, o tempo total até o `build-and-scan` é o do mais lento, não a soma dos três. Cada workflow bloqueia o commit de forma independente: se qualquer um falhar, o PR fica bloqueado mesmo que o `build-and-scan` já tenha concluído no `security.yml`.

Passo 6 — Fazer o push e verificar o pipeline completo

Faça o commit de todos os arquivos novos e modificados neste lab:

```
git add requirements.txt ruff.toml pytest.ini
.github/workflows/quality.yml .github/workflows/security.yml
git status
git commit -m "lab3: qualidade (ruff, cobertura) e observabilidade
(tracing otel)"
git push origin main
```

Após o push, acesse a aba **Actions** do repositório. Você verá agora três workflows iniciando simultaneamente: **Testes**, **Segurança** e **Qualidade**. O job `export-traces` aparecerá como o último a finalizar em `security.yml` mesmo quando `dast-smoke` foi pulado (PR em branch que não é `main`).

Para confirmar o comportamento do `export-traces` em falha:

```
git checkout -b feature/testar-traces
```

Edite `src/calculadora.py` e troque o valor de retorno de `calcular_desconto` para forçar falha nos testes unitários:

```
def calcular_desconto(preco: float, percentual: float) -> float:
    if percentual < 0 or percentual > 100:
        raise ValueError("Percentual deve estar entre 0 e 100")
    return round(preco * (1 - percentual / 100), 2) + 999 # erro
intencional
```

```
git add src/calculadora.py
git commit -m "test: forçar falha para observar export-traces"
git push origin feature/testar-traces
```

Abra o PR e observe: mesmo com jobs falhando, `export-traces` executa e produz a saída simulada, uqe é exatamente o comportamento esperado do `if: always()`.

Checklist de aceitação

Arquivos

- ☐ `ruff.toml` existe na raiz com as regras `E`, `F`, `W`
- ☐ `ruff` está no `requirements.txt`
- ☐ `pytest.ini` tem a seção `[coverage:run]` com `source = src.calculadora`
- ☐ `.github/workflows/quality.yml` existe com os jobs `lint-and-format` e `test-coverage`

- ☐ `.github/workflows/security.yml` tem o job `export-traces` com `if: always()`

Job `lint-and-format`

- ☐ `ruff check src/ tests/` — falha se houver erros de lint
- ☐ `ruff format --check src/ tests/` — falha se o código não estiver formatado
- ☐ Para verificar: introduza um import não usado em `calculadora.py` (`import os`) e confirme que o pipeline falha no step de lint

Job `test-coverage`

- ☐ Usa `--cov-fail-under=80` — falha o job se cobertura cair abaixo de 80%
- ☐ Publica `htmlcov/` como artefato — acessível na aba "Artifacts" da execução
- ☐ Para verificar: remova dois métodos de teste de `test_calculadora.py` e confirme que o pipeline falha por cobertura insuficiente

Job `export-traces`

- ☐ Declara `needs: [build-and-scan, dast-smoke]` e `if: always()`
- ☐ Roda mesmo quando `dast-smoke` foi pulado (PR em branch que não é `main`)
- ☐ Roda mesmo quando algum job anterior falhou

Pipeline completo

- ☐ Na aba Actions do repositório, os três workflows (`tests.yml`, `security.yml`, `quality.yml`) aparecem como execuções separadas e simultâneas para o mesmo commit
- ☐ O relatório HTML de cobertura (`coverage-report`) aparece como artefato — abra e verifique quais linhas não estão cobertas
- ☐ O job `export-traces` aparece como o último a completar em todas as execuções